

Annotated user question: What percentage of cards without power are in French?

Annotated external knowledge: in French refers to language = 'French'; cards without power refers to power IS NULL OR power = '*'; percentage = DIVIDE(COUNT(language = 'French' and power is NULL or power = '*'), COUNT(power is NULL or power = '*'))*100

Annotated SQL

```
SELECT CAST(SUM(CASE WHEN T2.language = 'French' THEN
  1 ELSE 0 END) AS REAL) * 100 / COUNT(T1.id)
FROM cards AS T1 INNER JOIN foreign_data AS T2 ON T1.
  uuid = T2.uuid
WHERE T1.power IS NULL OR T1.power = '*'
```

Schema

```
{"rulings": {"id": {"column_type": "integer", "column_description": "unique id identifying this ruling"}, ...}}
```

(a) Input example for SAR-Agent (BIRD Mini-Dev example 416).

Correctness: No – the annotated query counts one row per (card × language) after the join, so its denominator is inflated while the numerator only counts French rows, giving a much too small percentage.

Is_ambiguous: No – the question clearly asks for the share of cards whose power is NULL or "*" that have at least one French translation row.

Explanation

1. "Cards without power" → "power IS NULL OR power = '*'" in the 'cards' table.
2. We must count DISTINCT cards ('uuid') at this grain.
 - Total no-power cards: 31 053.
 - No-power cards with a French translation: 14 892.
3. Proper percentage = 14 892 / 31 053 ≈ 47.96 %.
4. The given query does an INNER JOIN to 'foreign_data', so each card is repeated for every language row; 'COUNT(T1.id)' therefore counts all languages, not cards. This mismatch makes the result (12.97 %) wrong.

Revised query:

```
SELECT 100.0 * COUNT(DISTINCT CASE WHEN fd.`language`
  = 'French' THEN c.`uuid` END) / COUNT(DISTINCT c
  .`uuid`) AS percentage_french
FROM `cards` AS c LEFT JOIN `foreign_data` AS fd
  ON c.`uuid` = fd.`uuid`
WHERE c.`power` IS NULL OR c.`power` = '*';
```

(b) diagnostic report for BIRD Mini-Dev example 416 generated by SAR-Agent.

Figure 12: Input to and output from SAR-Agent on BIRD Mini-Dev example 416.

A APPENDIX

A.1 Example

We present an end-to-end example illustrating how SAR-Agent verifies an annotation and generates a diagnostic report. Using problem 416 from the BIRD Mini-Dev set as an example (Figure 12a), SAR-Agent takes as inputs the annotation (the user query, relevant external knowledge, and the ground-truth SQL query) along with the database schema. The resulting diagnostic report is shown in Figure 12a. In addition, we show the agent’s reasoning trajectory in Figure 13.

Step 1

Explanation of the query: Count cards with no power or *

Verification query:

```
SELECT COUNT(*) FROM cards
WHERE power IS NULL OR power = '*'
```

Execution result: [(31053,)]

Step 2

Explanation of the query: Count rows after join all possible duplicates

Verification query:

```
SELECT COUNT(*) FROM (SELECT T1.uuid FROM cards T1
  LEFT JOIN foreign_data T2 ON T1.uuid=T2.uuid
  WHERE (T1.power IS NULL OR T1.power='*'))
```

Execution result: [(148569,)]

Step 3

Explanation of the query: Number of no-power cards that have French translation

Verification query:

```
SELECT COUNT(DISTINCT cards.uuid) FROM cards
JOIN foreign_data ON cards.uuid=foreign_data.uuid AND
  foreign_data.language='French'
WHERE cards.power IS NULL OR cards.power='*'
```

Execution result: [(14892,)]

Step 4

Explanation of the query: Compute annotated query result

Verification query:

```
SELECT CAST(SUM(CASE WHEN T2.language = 'French' THEN
  1 ELSE 0 END) AS REAL) * 100 / COUNT(T1.id) as
  percent
FROM cards AS T1 INNER JOIN foreign_data AS T2 ON T1.
  uuid = T2.uuid
WHERE T1.power IS NULL OR T1.power = '*' LIMIT 5
```

Execution result: [(12.975290140452374,)]

Step 5

Explanation of the query: manual ratio

Verification query:

```
SELECT 100.0*14892/31053
```

Execution result: [(47.95671915756932,)]

Step 6

Explanation of the query: Compute correct percentage

Verification query:

```
SELECT 100.0 * COUNT(DISTINCT CASE WHEN fd.language='
  French' THEN c.uuid END) / COUNT(DISTINCT c.uuid)
  AS pct
FROM cards c LEFT JOIN foreign_data fd ON c.uuid = fd.
  uuid
WHERE c.power IS NULL OR c.power='*' LIMIT 5
```

Execution result: [(47.95671915756932,)]

Figure 13: Trajectory of SAR-Agent on BIRD Mini-Dev example 416.

A.2 Configurations of Text-to-SQL Agents

We characterize the evaluated agents and their configurations in Table 8 based on the following six common modules.

- (1) *Fine-tuning*, including supervised fine-tuning (SFT) and reinforcement learning (RL), enhances LLMs’ ability to generate SQL queries.
- (2) *Question preprocessing* decomposes complex questions into simpler sub-questions or enriches user questions with the related database information.

- (3) *Schema linking* preprocesses the database schema for a user question. Techniques differ in their schema representations, in how they map the question to that representation, and in how they retrieve database values.
- (4) *SQL generation* produces candidate SQL queries using zero-shot and few-shot prompting; the latter augments the prompt with a few in-context examples.
- (5) *SQL refiner* revises generated queries based on execution feedback from the database to fix issues such as syntax errors or empty results.
- (6) *SQL selection* selects among multiple generated SQL candidates. The most common method is self-consistency, which chooses the query with the most agreement across sampled outputs. Other approaches include training a classifier for selection and generating unit-test queries for selection.

A.3 Correction Settings on the BIRD Dev Subset

We consider four correction settings using a randomly sampled 100-example subset of the BIRD Dev set. The settings are:

- (1) *Fully corrected Dev subset*: We corrected issues in the user question, external knowledge, ground-truth SQL, and database schema, and modified the database when necessary. The statistics for these corrections are shown in Table 4.
- (2) *Corrected Dev subset without database modification*: We corrected issues in the question, external knowledge, ground-truth SQL, and database schema, but left the database unchanged.
- (3) *SQL-only-corrected Dev subset by SAR-Agent*: We replaced only the misannotated ground-truth SQL with the repair generated by SAR-Agent. SAR-Agent produces fixed queries under the assumption that the user question and external knowledge have also been fixed. In this setting, however, the user question and external knowledge are left unchanged.
- (4) *SQL-only corrected Dev subset by humans*: We replaced only the misannotated ground-truth SQL with a human-repaired query. In this setting, we left the user question and external knowledge unchanged and aligned the repaired SQL with them, even if they contained errors.

We evaluated all 16 open-source text-to-SQL agents on the original (uncorrected) BIRD Dev subset and on four corrected Dev subsets. We report each agent’s execution accuracy and rankings on every subset in Table 9. CHESS [53], which ranks 7th on the original Dev subset, consistently ranks 1st across all four corrected subsets.

A.4 Case study

The Spider 2.0 team updated their user questions to resolve ambiguity on July 13, 2025. In this section, we study whether their fix addressed annotation errors.

Data settings. Among 121 examples with released gold queries, the Spider 2.0 team updated user questions in 71 examples. The updated set is called Spider 2.0-Snow, while the original set is called Spider 2.0-Snow-0713.¹¹

¹¹Spider 2.0-Snow can be found at: <https://github.com/xlang-ai/Spider2/blob/main/spider2-snow/spider2-snow.jsonl>, while Spider 2.0-Snow-0713 can be found at: <https://github.com/xlang-ai/Spider2/blob/main/spider2-snow/spider2-snow-0713.jsonl>

Old user question: Help me respectively calculate **the percentage of motorcycle accident fatalities** involving riders who were wearing helmets and those who weren’t?

Updated user question: ... 2) the percentage of motorcyclist fatalities in collisions where parties were not wearing helmets. For each group, compute this by **dividing the total number of motorcyclist fatalities by the total number of collisions** involving that group. ...

Figure 14: Example of an additional error after updating Spider 2.0-Snow problem sf_local015 by the Spider 2.0 team. The added formula calculates the number of fatalities per collision rather than the percentage of motorcyclist fatalities.

Evaluation method. We ran SAR-Agent on Spider 2.0-Snow-0713 to produce a diagnostic report for each example and manually verified the reported error reasons.

Evaluation results. We compare error rates for the original and updated versions of Spider 2.0-Snow. We find their correction reduces the error rate from 66.1% to 62.8%, implying that errors in only 3.3% of examples are fully resolved. This modest reduction indicates that the update by Spider 2.0 team has limited effectiveness.

We also find that their fix introduces additional errors in 6 examples.¹² For example, as illustrated in Figure 14, they added a formula that computes the number of fatalities per collision in the user question. However, this formula conflicts with the term “the percentage of motorcyclist fatalities” in the question. If some collisions result in more than one fatality, the resulting percentage of motorcyclist fatalities could exceed 100%, which is invalid. Moreover, because the corresponding gold SQL query multiplies the number of motorcyclist fatalities per collision by 100, even revising the question to “the number of motorcyclist fatalities per collision” would not resolve the error in this example. Therefore, only updating user questions is insufficient to fix the annotation errors in Spider 2.0-Snow. We advocate that future work refine the entire Spider 2.0-Snow benchmark, based on our proposed SAPAR, to improve annotation quality.

¹²Because these examples also contain other errors detected previously, the overall error rate of Spider 2.0-Snow is unchanged.

Table 8: Overview of the 16 open-source text-to-SQL agents from the BIRD leaderboard, including their respective models and modules. Abbreviations: RL = Reinforcement Learning; SFT = Supervised Fine-tuning.

Agents	Models	Modules					
		Fine-tuning	Question Preprocessing	Schema Linking	SQL Generation	SQL Refiner	SQL Selection
Contextual-SQL [1]	<i>Generator model:</i> Qwen2.5-Coder-32B-Instruct [49] <i>Reward model:</i> ctx-bird-reward-250121	×	×	×	few-shot	×	model-based
CSC-SQL [51]	<i>Merge model:</i> Qwen2.5-Coder-7B-Instruct [49] <i>Generation model:</i> XiYanSQL-QwenCoder-32B-2412 [20]	RL, SFT ¹	×	×	zero-shot	✓	self-consistency
GenaSQL [17]	text-embedding-3-small [39], GPT-4o-2024-08-06 [38] ²	×	×	✓	few-shot	✓	self-consistency, model-based
OpenSearch-SQL [60]	<i>Retrieval model:</i> bge-m3 [11] <i>Generation model:</i> GPT-4o-2024-05-13 [38]	×	×	✓	few-shot	✓	self-consistency
OmniSQL [30] ³	OmniSQL-32B	SFT	×	×	zero-shot	×	single candidate
CHESS [53] ⁴	GPT-4o-2024-08-06 [38]	×	×	✓	zero-shot	✓	test-based
Alpha-SQL [29]	<i>Retrieval model:</i> text-embedding-3-large [39] <i>Generation model:</i> Qwen2.5-Coder-32B-Instruct [49]	×	✓	✓	zero-shot	✓	self-consistency
GSR [3]	GPT-4o-2024-11-20 [38]	×	✓	✓	zero-shot	✓	single candidate
RSL-SQL [8]	GPT-4o-2024-08-06 [38]	×	×	✓	few-shot	✓	model-based
E-SQL [7]	GPT-4o-2024-08-06 [38]	×	✓	✓	few-shot	✓	single candidate
TA-SQL [48] ⁵	GPT-4o-2024-08-06 [38]	×	×	✓	few-shot	×	single candidate
DTS-SQL [45]	DeepSeek 7B [14]	SFT	×	✓	zero-shot	×	single candidate
MAC-SQL [56] ⁵	GPT-4o-2024-08-06 [38]	×	✓	✓	few-shot	✓	single candidate
SFT CodeS-15B [31]	SFT CodeS-15B	SFT	×	✓	zero-shot	×	single candidate
DAIL-SQL [18] ⁵	GPT-4o-2024-08-06 [38]	×	×	✓	few-shot	×	self-consistency
DIN-SQL [43] ⁵	GPT-4o-2024-08-06 [38]	×	✓	✓	few-shot	✓	single candidate

¹ Although CSC-SQL does not incorporate the SFT process, its SQL generation model, XiYanSQL-QwenCoder-32B-2412, is a fine-tuned model for text-to-SQL tasks.

² Since we do not have access to AWS Bedrock or Google Cloud, we use OpenAI text-embedding-3-small as our embedding model and GPT-4o-2024-08-06 for the other modules in GenaSQL.

³ We evaluate OmniSQL under the greedy decoding strategy.

⁴ We configure CHESS with three modules (Information Retriever, Candidate Generator, and Unit Tester) since this setup ranks highest on the BIRD leaderboard. Since we do not have access to Google Cloud, we run CHESS with GPT-4o instead.

⁵ Due to budget constraints, we run TA-SQL, MAC-SQL, DAIL-SQL, and DIN-SQL with GPT-4o instead of GPT-4.

Table 9: Execution accuracy and rankings of 16 open-source text-to-SQL agents on the original Dev subset (uncorrected) and four corrected Dev subsets.

Agents	Original Dev subset		Fully corrected Dev subset		Corrected Dev subset w/o database modification		SQL-only-corrected Dev subset by SAR-Agent		SQL-only-corrected Dev subset by humans	
	EX	Ranking	EX	Ranking	EX	Ranking	EX	Ranking	EX	Ranking
Contextual-SQL	73	1	71	7	74	7	63	12	69	13
OpenSearch-SQL	71	2	69	11	71	9	64	9	71	7
CSC-SQL	67	3	74	6	76	5	64	9	70	10
GSR	66	4	68	13	70	11	69	3	75	3
Alpha-SQL	65	5	78	3	80	3	71	2	76	2
GenaSQL	64	6	81	1	81	2	69	3	75	3
CHESS	62	7	81	1	83	1	76	1	80	1
E-SQL	62	7	70	8	73	8	68	5	75	3
OmniSQL	60	9	69	11	71	9	62	13	71	7
RSL-SQL	60	9	76	4	78	4	66	8	71	7
DTS-SQL	60	9	56	15	57	15	53	15	58	15
TA-SQL	59	12	70	8	70	11	67	7	74	6
DIN-SQL	59	12	70	8	70	11	64	9	70	10
MAC-SQL	58	14	75	5	75	6	68	5	70	10
DAIL-SQL	56	15	68	13	67	14	58	14	63	14
SFT CodeS-15B	56	15	52	16	53	16	52	16	58	15